

SAGA: A Memory-Efficient Accelerator for GANN Construction via Harnessing Vertex Similarity

Ruiyang Chen, Xueyuan Liu, Chunyu Qi, Yuanzheng Yao, Yanan Sun, Xiaoyao Liang, and Zhuoran Song*
Shanghai Jiao Tong University, Shanghai, China
{chenruiyang, songzhuoran}@sjtu.edu.cn

Abstract—Graph-traversal-based Approximate Nearest Neighbor (GANN) search and construction have become key retrieval techniques in various domains, such as recommendation systems and social networks. However, deploying GANN in real-world scenarios faces significant challenges, as high-dimensional vertices within the graph can lead to intensive memory demands. Although architectures like NDSearch have been proposed to accelerate GANN search, they are hard to deploy for GANN construction, as their pre-processing methods introduce massive overhead in dynamic graphs. In this paper, given the observation that neighboring vertices in a dynamic graph exhibit feature similarity, we propose SAGA, the first accelerator that alleviates memory bound in GANN construction. To capture this similarity, we directly leverage the first step of construction to gather vertices with the same starting point into a cluster to minimize the similarity detection overhead. Next, we decompose vertices into key and non-key ones, where their deltas fall in a narrow range, which is suitable to be quantized to lower bit widths. Building upon this approach, we design a specialized architecture, which efficiently implements the GANN construction by two-level scheduling and a mixed-precision supported bit-serial unit. Through comprehensive evaluation, we demonstrate that SAGA can achieve an average speedup of 9.30 $\times$, 4.87 $\times$, 4.15 $\times$ and 35.46 $\times$, 7.60 $\times$, 5.15 $\times$ energy savings over CPU, GPU and NDSearch, respectively, while retaining task accuracy.

I. INTRODUCTION

Graph-traversal-based Approximate Nearest Neighbor (GANN) construction and search have become critical techniques in various domains, including recommendation systems [1], [2], computer vision [3], [4], and social networks [5], [6]. In these applications, GANN first performs construction to generate a graph that can guide searches, after which it searches the constructed graph to find the top-K nearest vertices to a given query. Unfortunately, considering that high-dimensional vertices consume substantial memory, the deployment of GANN in real-world scenarios is significantly constrained by memory bottlenecks [7]–[13]. Moreover, since the graph is **dynamically** updated during construction but remains **static** during searching, the construction stage is more complex. *Therefore, the goal of this work is to alleviate the memory bottleneck in GANN construction.*

* Zhuoran Song is the corresponding author.

This work is partly supported by the National Natural Science Foundation of China (Grant No. 62202288), the Natural Science Foundation of Shanghai (#23ZR1433200), and the Shanghai 2024 “Science and Technology Innovation Action Plan” Fundamental Research Program in Integrated Circuit (No. 24JD1400300).

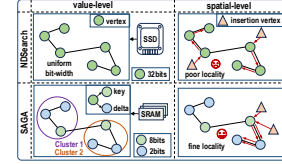


Fig. 1. Comparison of NDSearch [15] and SAGA.

To address the memory-bound problem of GANN, many existing works [14], [15] focus on accelerating GANN search. Taking NDSearch [15] as an example, it involves pre-processing the static graph by remapping the neighboring vertices on the same page of SSD to improve the search locality. However, since the graph is dynamically updated during GANN construction, deploying NDSearch in construction requires iteratively executing its pre-processing method at runtime, incurring significant latency and making GANN search accelerators ill-suited for construction. In contrast, our approach leverages the unique characteristics of GANN to accelerate the construction process. **Inspired by the homophily theory [16] in social networks, where users within the same community tend to share similar characteristics such as ages or interests, we discovered that neighboring vertices exhibit feature similarity. This similarity allows us to reduce precision in representing the differences between similar features, thus alleviating the memory bandwidth stress.** For example, given two vertices (20, 18, 10, 9) and (20, 17, 9, 8) and their difference (0, 1, 1, 1), naively storing each vertex takes 4 Bytes, while storing their difference only takes 1 Byte, saving 75% of memory storage.

In this paper, we propose **the first accelerator for GANN construction**, named SAGA. We observe that the first step in construction can naturally indicate vertices’ similarity, so we carefully leveraged the intermediate results from this step to gather vertices into clusters and decompose vertices into one key vertex and multiple non-key vertices. To take advantage of value-level similarity within each cluster, we differentiate non-key vertices from key ones to construct deltas, which fall in a narrower range compared to the original vertices, making them more suitable for low-precision representation. Moreover, given that similar vertices are more likely to access the same vertex during graph traversal, we propose to exploit such a spatial-level similarity by rearranging the access order of vertices. To further illustrate the superiority of SAGA, Fig. 1 provides a comparison between NDSearch and our

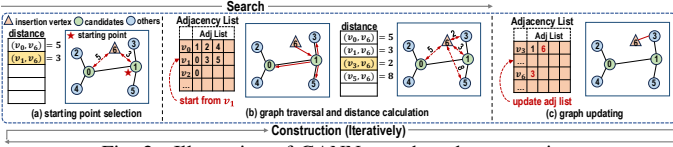


Fig. 2. Illustration of GANN search and construction.

approach. At the value level, NDSearch stores vertices in SSD with full precision, whereas we use a reduced bit width for delta storage, making GANN fit within on-chip SRAM. At the spatial level, NDSearch constructs vertices in a random order, while we reorder the execution sequence of similar vertices, thus achieving better locality.

Our contributions can be summarized as follows:

- From the algorithm perspective, we propose a similarity detection algorithm that gathers similar vertices into clusters. Then we introduce a quantization algorithm that leverages vertex similarity to reduce storage bit width.
- From the hardware perspective, we present an architecture design and develop a two-level scheduling that optimizes both the insertion and access order of vertices.

II. BACKGROUND

A. GANN Construction and Search

GANN construction consists of three steps: starting point selection, distance calculation, and graph updating. First, starting point selection is used to reduce redundant calculations during construction. To simplify the search space, GANN samples certain vertices from the vertex set as candidates for the starting point, corresponding to vertices v_0 and v_1 in Fig. 2(a). Then, it calculates the distance from insertion vertex v_6 to these candidates and selects the closest candidate as the starting point. Typically, the distance is measured using either Inner Product (IP) or Euclidean (L2) distance, where the mathematical forms are $v_i \cdot v_j$ and $(v_i - v_j)^2$, respectively, with v_i and v_j being the vertices in the calculation. Second, distance calculation computes the distance between the insertion vertex and all neighbors of the starting point. In Fig. 2(b), v_6 sequentially computes distances to v_0 , v_3 , and v_5 , which are 5, 2, and 8. Thus, the closest vertex to v_6 is identified as v_3 . Notably, distance computation is iterative; if the neighbors of the starting point also have neighbors, they will also be calculated until the algorithm's pre-defined conditions are met. Finally, graph updating establishes a bi-directional link between the insertion vertex and its top-K closest neighbor. In Fig. 2(c), a bi-directional link is added between v_6 and v_3 , and the adjacency list is then updated.

Given a constructed graph, GANN search uses the starting point selection and distance calculation steps to find the top-K nearest vertices to a given query, as shown in Fig. 2. Note that during the search process, the graph is completely static, while during construction, the graph is dynamic.

B. Related Works

There is a series of works on accelerating GANN [14], [15], but they all focus on searching rather than construction. SmartSSD [14] is the first architecture to accelerate GANN

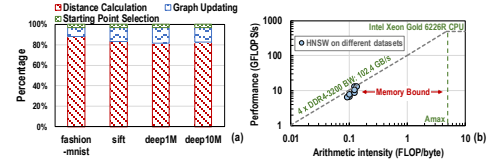


Fig. 3. (a) Execution time breakdown of GANN construction; (b) Roofline model in distance calculation.

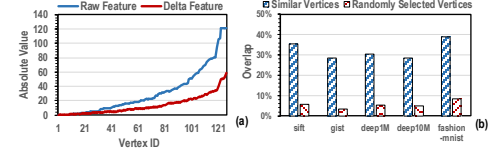


Fig. 4. (a) Value distribution in ascending order; (b) Graph traversal overlap.

search. To make graph search algorithms more efficient on their hardware, SmartSSD pre-partitions the original static graph offline and performs searching separately on each sub-graph, narrowing the search scope. NDSearch [15] is the state-of-the-art accelerator for GANN search. NDSearch proposes remapping vertices on the SSD so that the neighboring vertices of each vertex are physically adjacent. To determine such remapping order, NDSearch conducts an expensive breath-first search (BFS) traversal of the entire graph during pre-processing. Unfortunately, both of their pre-processing approaches need to be iteratively executed whenever the graph is changed, which frequently happens during construction. As a result, their methods will introduce significant latency and may not be the proper option for GANN construction.

III. MOTIVATION

To find the performance bottlenecks in GANN construction, we conducted a breakdown analysis of the construction process on a CPU, which reveals that the majority of the execution time originates from distance computations, as shown in Fig. 3(a). Following this, we present our roofline model experiment for the distance calculation step in Fig. 3(b), demonstrating that all results fall into the left side of the roofline, which is in the DRAM bandwidth-constrained region. This sub-optimal performance is mainly due to the high dimension of vertices and their irregular memory accesses. This motivates us to find a solution that optimizes both storage and locality problems during distance calculations.

Observation 1: Vertex features contain redundancy at the value level. Fig. 4(a) visualizes the differences between neighboring vertices on sift [17] dataset. Compared to vertices' raw features, deltas have a narrower distribution, making them better suited for low-precision representation. Based on this observation, we propose quantizing deltas to a lower bit width, thus improving bandwidth utilization.

Observation2: Graph traversal exhibit reuse at the spatial level. As shown in Fig. 4(b), we compare the proportion of graph traversal overlap for similar vertices, and the proportion of graph traversal overlap for randomly selected vertices. The comparison between them shows that similar vertices are more likely to access the same vertex. This motivates us to bundle similar vertices together to enhance data locality.

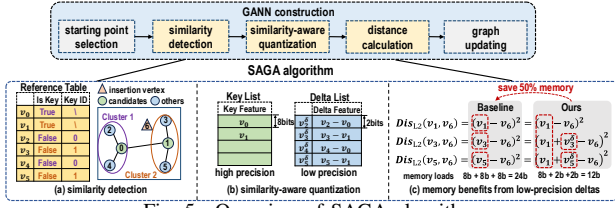


Fig. 5. Overview of SAGA algorithm.

IV. SAGA ALGORITHM

In this section, we introduce our SAGA algorithm, which comprises three essential operators: similarity detection, similarity-aware quantization and distance calculation.

A. Similarity Detection

To capture the similarity between vertices, we propose leveraging the intermediate results from the starting point selection step. This proposal builds on a well-known theory that users owning similar features in the community tend to connect through hubs [16]. Specifically, we find that candidates from construction's first step are naturally suited to serve as hubs, where vertices selecting the same candidate as a starting point are highly likely to be similar.

To implement similarity detection, we record the starting point of each insertion vertex and gather vertices with the same starting point into a single cluster. We designate this starting point as the **key** vertex, where other vertices are considered **non-key** vertices. To avoid duplicate storage of the same key vertex, each of the non-key vertices needs to record its key vertex's ID in a Reference Table. Fig. 5(a) shows an example of similarity detection: v_2 and v_4 both share v_0 as their starting point, so they are gathered into a cluster with v_0 as the key. In the Reference Table, we record whether each vertex is a key vertex (Is Key) and its corresponding key vertex's ID (Key ID). For instance, since v_2 is a non-key vertex, the Reference Table marks its Is Key identifier as False and records its Key ID as 0, allowing v_2 to directly reference its key vertex v_0 .

B. Similarity-Aware Quantization

After identifying the similarities, we design a similarity-aware quantization algorithm to reduce storage bit width, which is a pre-processing step for distance calculations. Given the Reference Table and a non-key vertex v_i , we look up the key vertex v_j in the Reference Table and calculate the delta v_i^δ for v_i as follows: $v_i^\delta = v_i - v_j$. Next, we apply high-precision quantization to v_j with a wide range, while using low-precision quantization to v_i^δ with a narrow range. Fig. 5(b) illustrates our method: for key vertex v_0 , we store its feature with 8-bit in the Key List; for non-key vertices, such as v_2 , we compute its delta from v_0 and store it with 2-bit in the Delta List, recorded as v_2^δ . It is important to note that since there are numerous delta vertices but few key vertices, storing delta vertices with low precision effectively reduces the required bandwidth. Fig. 5(c) shows a specific example: with low-bit storage for deltas, the three distance calculations in our method require only 12 bits of memory loads, while the baseline approach requires 24 bits for the same calculations. Our method reduces memory overhead by 50%.

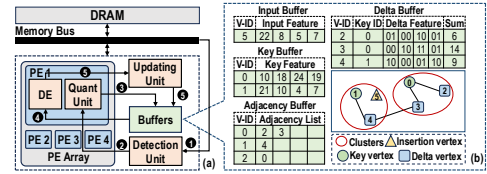


Fig. 6. Overview of (a) SAGA architecture; (b) Buffers (V-ID: Vertex ID).

C. Distance Calculation

Since we apply different bit widths for key and non-key vertices, the original distance calculation is no longer applicable, so we will discuss how to achieve distance calculation between insertion vertex v_{in} and both key and non-key vertices.

1. For key vertex v_j , we directly compute its distance to v_{in} by IP ($Dis_{IP}(v_{in}, v_j)$) or L2 ($Dis_{L2}(v_{in}, v_j)$). We store the distance in a partial sum table, indexed by v_j 's ID, which will be used to avoid redundant computations for the same v_j .
2. For the non-key vertex v_i , the equations for the IP and L2 calculations are as follows:

$$Dis_{IP}(v_{in}, v_i) = Dis_{IP}(v_{in}, v_j) + Dis_{IP}(v_{in}, v_i^\delta) \quad (1)$$

$$Dis_{L2}(v_{in}, v_i) = Dis_{L2}(v_{in}, v_j) - 2Dis_{IP}(v_{in} - v_j, v_i^\delta) + Dis_{IP}(v_i^\delta, v_i^\delta) \quad (2)$$

From the formulas, we see that the number of distance calculations increases from 1 to 2 (for the IP) and 3 (for the L2), which might seem to negate performance gains. Fortunately, we find that the distance calculations between v_{in} and v_i can be saved by reusing the distance between v_{in} and v_j ($Dis_{IP}(v_{in}, v_j)$ or $Dis_{L2}(v_{in}, v_j)$). Therefore, we retrieve $Dis_{IP}(v_{in}, v_j)$ or $Dis_{L2}(v_{in}, v_j)$ from the partial sum table and add it to the remaining terms in Eqn. 1 and Eqn. 2 to obtain the final result. Additionally, $Dis_{IP}(v_i^\delta, v_i^\delta)$ in the L2 distance calculation is also a constant value for each v_i^δ and can also be pre-computed during quantization.

D. Walk-Through Example

Fig. 5 also provides an example of how the SAGA algorithm integrates with the original construction process. Steps that remain unchanged are marked in grey, while steps improved by SAGA are highlighted in yellow. First, we perform the starting point selection, a step that remains unmodified. Next, SAGA leverages the results of the starting point selection for similarity detection, which in turn guides the similarity-aware quantization. Given the quantized vertices, SAGA calculates the distance based on whether a vertex is a key or a non-key vertex. Finally, the graph is updated based on the calculated distances. The above steps are repeated until all insertion vertices have been fully processed.

V. PROPOSED HARDWARE DESIGN

A. Overview

To support our SAGA algorithm, we co-design a hardware architecture. Fig. 6(a) illustrates the overall architecture of SAGA, which includes several on-chip buffers, a Detection

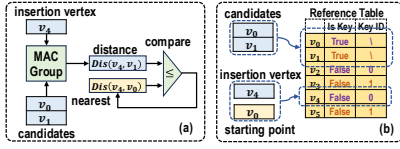


Fig. 7. Detection Unit for (a) starting point selection; (b) similarity detection.

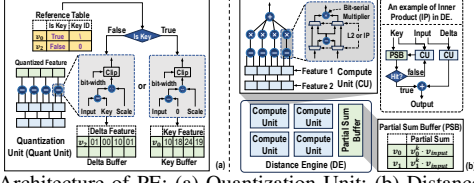


Fig. 8. Architecture of PE: (a) Quantization Unit; (b) Distance Engine.

Unit, an array of Processing Elements (PEs), and an Updating Unit. Each PE comprises a Quantization Unit (Quant Unit) and a Distance Engine (DE).

Dataflow. The dataflow in SAGA is also shown in Fig. 6(a), while the detailed buffer entries are illustrated in Fig. 6(b). ❶ To select the starting point, a batch of insertion vertices is transferred from DRAM to the Detection Unit via the memory bus. The Detection Unit then searches for the nearest starting point for each insertion vertex. ❷ In order to capture the similarity between vertices, the Detection Unit leverages the starting point information to generate a Reference Table, which stores whether each vertex is a key vertex and its Key ID. ❸ The Quant Unit performs quantization separately for key and non-key vertices and stores the features in Key Buffer and Delta Buffer, respectively. ❹ The DE retrieves the quantized features from Input Buffer, Key Buffer and Delta Buffer to compute distances. ❺ To update the constructed graph, the Updating Unit reads the distances computed by the DE, selects the top-K nearest vertices to the insertion vertex, and establishes bi-directional links in the Adjacency Buffer. The loop ❶ to ❺ repeats until the construction terminates.

B. Detection Unit

The Detection Unit is responsible for executing the starting point selection and similarity detection steps during the construction process. Fig. 7(a) illustrates how the Detection Unit selects starting points: given an insertion vertex, the Detection Unit uses a MAC group to compute its distance to all candidates and applies a comparator to select the nearest candidate as the insertion vertex's starting point. Fig. 7(b) depicts the similarity detection process. For candidates, the Detection Unit marks their Is Key identifiers as True. For insertion vertices, the Detection Unit sets their Is Key identifiers to False and records their starting point's ID as Key ID.

C. Processing Engine Array

The PE array of SAGA contains four PEs, each PE consists of a Quantization Unit and a Distance Engine.

Quantization Unit. The Quantization Unit (Quant Unit) is responsible for determining whether a vertex is a key vertex and performing its quantization. Fig. 8(a) depicts the architecture of the Quant Unit. The Quant Unit first determines whether a vertex is a non-key or key vertex based on the Is

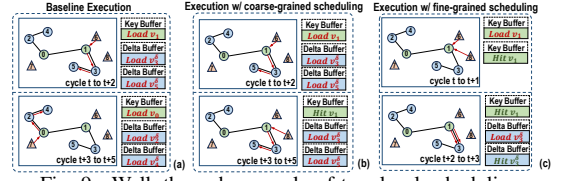


Fig. 9. Walk-through example of two-level scheduling.

Key identifier recorded in the Reference Table. The Quant Unit then performs quantization and writes the quantized features into the corresponding buffer.

Distance Engine. The Distance Engine (DE) is the most important component within the PE, responsible for computing the IP/L2 distances. As depicted in Fig. 8(b), each DE includes four Compute Units (CUs) dedicated to distance calculations and one Partial Sum Buffer (PSB). Each CU accepts two feature vectors as input and performs distance calculations based on the chosen metric (IP or L2); the PSB functions as a multi-way set-associative cache for storing reusable partial sums. Generally, a DE can simultaneously compute the distances for one insertion vertex and four neighboring vertices.

Based on the quantization algorithm outlined in Section IV-C, we will detail the distance calculations for both key and non-key vertices.

Key Vertex. When the insertion vertex requires a distance calculation with a key vertex, we retrieve v_{in} from the Input Buffer and v_j from the Key Buffer, then send them to the CU for the computation of the partial sum. Since vertices within the same cluster can reuse the partial sum, we store its value in the PSB and index it by the Key ID. If there is insufficient space in the PSB to accommodate the new partial sum, we identify an entry to evict according to the Least Recently Used (LRU) policy and replace it with the new partial sum.

Non-Key Vertex. According to Eqn. 1 and Eqn. 2, we trigger DE to compute $Dis_{IP}(v_{in}, v_i^\delta)$ or $Dis_{IP}(v_{in} - v_j, v_i^\delta)$. As for partial sum ($Dis_{IP}(v_{in}, v_j)$ or $Dis_{L2}(v_{in}, v_j)$), we first look it up in the PSB using the Key ID as an index. If it hits in PSB, we can retrieve the corresponding value; if it misses in PSB, we will compute the partial sum and store it in the PSB. At last, regarding the $Dis_{IP}(v_i^\delta, v_i^\delta)$ in Eqn. 2, we directly index it in the Sum column of Delta Buffer with V-ID.

Since key and delta features differ in bit width, we employ a bit-serial method in the CU to leverage the computational advantages of low bit width representation.

D. Updating Unit

The Updating Unit is used for sorting the distances and updating the graph accordingly. The Updating Unit receives the output from the DE and produces the top-K vertices with the smallest distances. To achieve this, Updating Unit employs a hardware implementation similar to that in [18], maintaining a minimum heap for up to one hundred input distances. After obtaining the top-K nearest vertices, the Updating Unit updates the corresponding entries in the Adjacency Buffer, completing the graph construction process.

E. Two-level Scheduling

In our previous design, we reduced storage and computational costs by lowering the bit width. However, due to the irregular memory access in construction, Key/Delta Buffers still encounter a high miss rate. Therefore, we propose a two-level scheduling that reorders vertex from both coarse-grained and fine-grained perspectives.

The coarse-grained scheduling rearranges the insertion order of the vertices, ensuring that vertices within the same cluster are executed sequentially. This is justified by the overlapping graph traversal paths of vertices in the same cluster. After coarse-grained scheduling, the fine-grained scheduling continues to reorder the access order of vertices. Considering that the reusable entries of insertion vertices in the Key/Delta Buffers may be evicted due to capacity constraints, we propose interleaving the distance computation for insertion vertices.

We then provide a walk-through example to illustrate the benefits of applying two-level scheduling. Fig. 9(a) shows the situation without scheduling, revealing poor locality in the Key/Delta Buffers. Fig. 9(b) illustrates the result of coarse-grained scheduling, where we reordered v_7 and v_8 . This allows similar vertices v_6 and v_8 to be executed sequentially. It is evident that the coarse-grained scheduling strategy transforms v_8 's access to v_1 from a miss to a hit, thereby improving the locality of the Key Buffer. However, due to the constrained capacity of the Delta Buffer, reusable entries (v_3^δ and v_5^δ) still get evicted, leading to a high miss rate in the Delta Buffer. Fig. 9(c) demonstrates the effect of applying both coarse-grained and fine-grained scheduling, where we interleave the distance calculations for v_6 and v_8 , allowing for reuse of results in the Delta Buffer. With fine-grained scheduling, we make v_1 hit in Key Buffer and v_3^δ hit in Delta Buffer, demonstrating fine locality in both the Delta and Key Buffers.

VI. EVALUATION

A. Experiment Methodology

Benchmark and datasets. We evaluated SAGA on HNSW [12], the most representative and widely cited GANN algorithm. The evaluation was conducted on five datasets from the ANN-benchmark [19]: sift [17], deep1M [20], deep10M [20], fashion-mnist [21], and gist [22]. For each dataset, we use the recommended parameter settings and distance metrics from the ANN-benchmark during construction.

Experiment Setup. To model the behavior of SAGA, we build a cycle-accurate simulator. We implement and synthesize the digital logic circuits at the 28nm technology node with 500MHz using Synopsys Design Compiler. The on-chip buffers are evaluated using CACTI 7.0 [23] with a 32nm technology node, and we use the DeepScaleTool [24] to scale the results down to 28nm. For CPU and GPU, we selected FAISS [25] and cuhns [26] libraries as implementations of HNSW. We use an Intel Xeon Gold 6226 CPU running at 2.0GHz as the baseline for the CPU, and an A100 GPU with 80GB VRAM as the baseline for the GPU. To compare SAGA with previous GANN accelerators, we selected the state-of-the-art NDSearch [15] as a baseline. As GANN iteratively

TABLE I
COMPARISONS WITH NDSEARCH (FP32) AND SAGA (CR:
COMPRESSION RATIO).

Dataset	Config	Accuracy	Average Bits	CR
sift [17]	NDSearch	96.0 $\pm$ 2.0%	32	1 $\times$
	SAGA	94.2 $\pm$ 2.1%	2.19	14.63 $\times$
gist [22]	NDSearch	94.7 $\pm$ 2.0%	32	1 $\times$
	SAGA	93.4 $\pm$ 2.4%	2.07	15.48 $\times$
deep1M [20]	NDSearch	95.3 $\pm$ 2.0%	32	1 $\times$
	SAGA	94.2 $\pm$ 2.4%	2.36	13.52 $\times$
deep10M [20]	NDSearch	95.9 $\pm$ 1.6%	32	1 $\times$
	SAGA	94.6 $\pm$ 2.1%	2.43	13.18 $\times$
fashion-mnist [21]	NDSearch	97.8 $\pm$ 0.9%	32	1 $\times$
	SAGA	96.1 $\pm$ 1.4%	2.29	13.97 $\times$

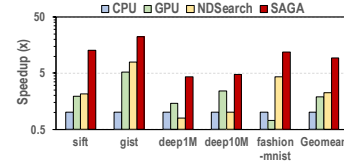


Fig. 10. The speedup of SAGA over CPU, GPU and NDSearch.

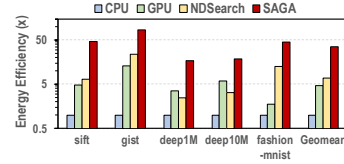


Fig. 11. The energy efficiency of SAGA over CPU, GPU and NDSearch.

updates the graph during construction, the BFS introduced by NDSearch must be re-executed after each update. Since NDSearch does not incorporate additional hardware to support BFS, we implemented BFS on the CPU using the high-performance kernel from GAP [27]. To ensure a fair comparison, we scaled NDSearch's compute units and buffers under the same area budget as SAGA.

B. Accuracy

In TABLE I, we present a comparison of accuracy and compression ratio between our method and NDSearch across various datasets. Extensive experiments demonstrate that our approach achieves a high compression ratio while maintaining accuracy. We leveraged the value-level similarity between vertices by using high-bitwidth representations for critical key vertices, while employing low-bitwidth representations for the majority of non-key vertices. This approach achieves a well-balanced trade-off between accuracy and compression ratio.

C. Overall Results

Speedup Fig. 10 illustrates the speedup of SAGA. Generally, SAGA achieves an average speedup of 9.30 $\times$, 4.87 $\times$ and 4.14 $\times$ over CPU, GPU and NDSearch. In comparison with CPU, SAGA manifests the benefits of customized architecture, which offers higher bandwidth utilization and better on-chip buffer locality. Compared to GPU, although they offer high throughput, GANN construction is not a compute-centric application, which leads to low utilization of the GPU cores. For example, in the fashion-mnist dataset, we observe that the GPU performance is even lower than the CPU. Additionally, SAGA outperforms the state-of-the-art GANN accelerator NDSearch because our quantization method achieves a higher

TABLE II
AREA AND POWER COMPARISON BETWEEN SAGA AND NDSEARCH.

	SAGA		
	Config	Area (mm^2)	Power (mW)
Detection Unit	1×(32-bit)	0.19	5.51
Distance Engine	4×(bit-serial)	0.35	97.73
Quant Unit	4×(8-bit)	0.07	21.72
Updating Unit	1×(8-bit)	0.03	5.43
On-chip Buffers	320KB	0.62	197.34
Overall	-	1.26	327.73

	NDSearch		
	Config	Area (mm^2)	Power (mW)
Detection Unit	1×(32-bit)	0.19	5.51
Distance Engine	512 MACs(32-bit)	0.58	173.75
Quant Unit	-	-	-
Updating Unit	1×(32-bit)	0.12	21.71
On-chip Buffers	320KB	0.64	202.40
Overall	-	1.53	403.37

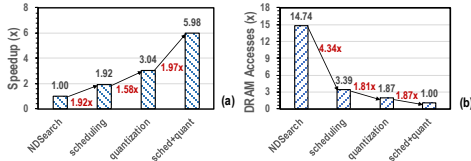


Fig. 12. The contribution of each method to (a) the improvement of performance and (b) the DRAM access reduction on the sift dataset.

compression ratio (see Table I). Moreover, we observe that the BFS operations performed by NDSearch introduce significant overhead. In the deep1M experiment, NDSearch even fails to outperform the CPU baseline, with BFS accounting for 77% of its execution time. Therefore, NDSearch is not well-suited for dynamic construction processes.

Energy Efficiency. Fig. 11 shows the energy efficiency of SAGA. For energy efficiency, SAGA offers an average energy efficiency of $35.48\times$, $7.60\times$ and $5.15\times$ over CPU, GPU and NDSearch. Thanks to SAGA’s quantization algorithm, it achieves higher bandwidth utilization, significantly reducing the overall DRAM access time. Additionally, SAGA’s scheduling algorithm enhances the locality of on-chip buffers and further reduces the volume of DRAM read and write, leading to substantial energy savings in memory access.

Area and Power. The area and power consumption of SAGA are summarized in Table II. SAGA utilizes only a small number of computational engines and units, as GANN construction is not a compute-intensive task. Compared to NDSearch, SAGA maintains the same buffer capacity but can store more data due to its lower bit width representations, significantly enhancing locality. Note that SAGA achieves a 17.65% reduction in area and an 18.75% reduction in power compared to NDSearch. This is attributed to SAGA’s low-bitwidth hardware design, whereas NDSearch is designed for full precision.

D. Detailed Analysis

Ablation study. We conduct the ablation study on the proposed methods, including two-level scheduling (*scheduling*), similarity-aware quantization (*quantization*) and the combi-

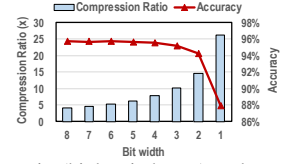


Fig. 13. Compression ratio (higher is better) and accuracy (higher is better) compared with NDSearch under different bit widths on the sift dataset.

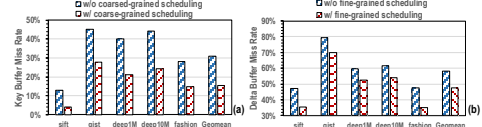


Fig. 14. The effect of scheduling in (a) Key Buffer miss rate (lower is better) and (b) Delta Buffer miss rate (lower is better).

nation of scheduling and quantization (*sched+quant*), which is the final version of SAGA. Fig. 12(a) illustrates the performance improvements achieved by SAGA’s methods. As shown in the figure, SAGA achieves a $1.92\times$ speedup due to enhanced buffer locality enabled by *scheduling*. SAGA also achieves a $3.04\times$ speedup by leveraging lower bit width for storage using *quantization*. Finally, when combining *scheduling* and *quantization*, SAGA achieves a remarkable $5.98\times$ speedup. Fig. 12(b) further demonstrates SAGA’s improvement in DRAM accesses, our proposed methods achieve DRAM reductions of $4.34\times$, $7.88\times$ and $14.74\times$, respectively.

Parameter Trade-off. In SAGA, we need to determine the bit width for deltas to balance the compression ratio and accuracy. Note that the key vertex is represented using 8-bit. As shown in Fig. 13, when the bit width decreases, SAGA achieves higher compression ratio, reaching up to $26.26\times$ when the bit width is 1. However, the accuracy in this case drops significantly to 87.9%, which is unacceptable. Therefore, we select 2-bit for deltas, achieving a compression ratio of $14.63\times$ while maintaining a high accuracy of 94.2%.

Scheduling. To evaluate the two-level scheduling, we explore the impact of coarse/fine-grained scheduling on the Key/Delta Buffers, respectively. Fig. 14 verifies that both coarse/fine-grained scheduling can effectively reduce miss rates of the two buffers. This improvement is due to the enhanced temporal locality of vertex accesses.

VII. CONCLUSION

In this paper, we propose SAGA, the first accelerator to leverage similarity to address the memory-bound challenges in GANN construction. To capture similarity, we first introduce a similarity detection algorithm that clusters vertices based on their starting points. Next, to reduce storage overhead, we propose a similarity-aware quantization algorithm that quantizes the deltas of similar vertices to lower bit width. We design a hardware architecture and develop a two-level scheduling algorithm to optimize locality in buffers. Experimental results demonstrate that SAGA achieves an average $4.15\times$ speedup and $5.15\times$ energy efficiency compared to the state-of-the-art NDSearch, with negligible accuracy loss.

REFERENCES

- [1] Y. Park, S. Park, W. Jung, and S.-g. Lee, "Reversed cf: A fast collaborative filtering algorithm using a k-nearest neighbor graph," *Expert Systems with Applications*, vol. 42, no. 8, pp. 4022–4028, 2015.
- [2] Y. Meng, X. Dai, X. Yan, J. Cheng, W. Liu, J. Guo, B. Liao, and G. Chen, "Pmd: An optimal transportation-based user distance for recommender systems," in *Advances in Information Retrieval: 42nd European Conference on IR Research, ECIR 2020, Lisbon, Portugal, April 14–17, 2020, Proceedings, Part II* 42. Springer, 2020, pp. 272–280.
- [3] J. Wang, J. Wang, G. Zeng, Z. Tu, R. Gan, and S. Li, "Scalable k-nn graph construction for visual descriptors," in *2012 IEEE Conference on Computer Vision and Pattern Recognition*. IEEE, 2012, pp. 1106–1113.
- [4] A. Kosuge and T. Oshima, "An object-pose estimation acceleration technique for picking robot applications by using graph-reusing k-nn search," in *2019 First International Conference on Graph Computing (GC)*. IEEE, 2019, pp. 68–74.
- [5] N. Chen, "On the approximability of influence in social networks," *SIAM Journal on Discrete Mathematics*, vol. 23, no. 3, pp. 1400–1415, 2009.
- [6] S. Kumar and P. Kumar, "Upper approximation based privacy preserving in online social networks," *Expert Systems with Applications*, vol. 88, pp. 276–289, 2017.
- [7] C. Fu, C. Wang, and D. Cai, "High dimensional similarity search with satellite system graph: Efficiency, scalability, and unindexed query compatibility," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 8, pp. 4139–4150, 2021.
- [8] A. Singh, S. J. Subramanya, R. Krishnaswamy, and H. V. Simhadri, "Freshdiskann: A fast and accurate graph-based ann index for streaming similarity search," *arXiv preprint arXiv:2105.09613*, 2021.
- [9] C. Fu, C. Xiang, C. Wang, and D. Cai, "Fast approximate nearest neighbor search with the navigating spreading-out graph," *arXiv preprint arXiv:1707.00143*, 2017.
- [10] Y. Malkov, A. Ponomarenko, A. Logvinov, and V. Krylov, "Approximate nearest neighbor algorithm based on navigable small world graphs," *Information Systems*, vol. 45, pp. 61–68, 2014.
- [11] W. Dong, C. Moses, and K. Li, "Efficient k-nearest neighbor graph construction for generic similarity measures," in *Proceedings of the 20th international conference on World wide web*, 2011, pp. 577–586.
- [12] Y. A. Malkov and D. A. Yashunin, "Hnsw: Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2020.
- [13] M. Wang, X. Xu, Q. Yue, and Y. Wang, "A comprehensive survey and experimental comparison of graph-based approximate nearest neighbor search," *Proc. VLDB Endow.*, vol. 14, no. 11, p. 1964–1978, Jul. 2021. [Online]. Available: <https://doi.org/10.14778/3476249.3476255>
- [14] J.-H. Kim, Y.-R. Park, J. Do, S.-Y. Ji, and J.-Y. Kim, "Accelerating large-scale graph-based nearest neighbor search on a computational storage platform," *IEEE Transactions on Computers*, vol. 72, no. 1, pp. 278–290, 2022.
- [15] Y. Wang, S. Li, Q. Zheng, L. Song, Z. Li, A. Chang, Y. Chen *et al.*, "Nd-search: Accelerating graph-traversal-based approximate nearest neighbor search through near data processing," in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2024, pp. 368–381.
- [16] M. Mcpherson, L. S. Lovin, and J. Cook, "Birds of a feather: Homophily in social networks," *Annual Review of Sociology*, 2001.
- [17] J. Brandt, "Transform coding for fast approximate nearest neighbor search in high dimensions," in *2010 IEEE Computer Society Conference on Computer Vision and Pattern Recognition*. IEEE, 2010, pp. 1815–1822.
- [18] R. Bhagwan and B. Lin, "Fast and scalable priority queue architecture for high-speed network switches," in *Proceedings IEEE INFOCOM 2000. Conference on Computer Communications. Nineteenth Annual Joint Conference of the IEEE Computer and Communications Societies (Cat. No. 00CH37064)*, vol. 2. IEEE, 2000, pp. 538–547.
- [19] M. Aumüller, E. Bernhardsson, and A. Faithfull, "Ann-benchmarks: A benchmarking tool for approximate nearest neighbor algorithms," *Information Systems*, vol. 87, p. 101374, 2020.
- [20] A. Babenko and V. Lempitsky, "Efficient indexing of billion-scale datasets of deep descriptors," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 2055–2063.
- [21] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms," *arXiv preprint arXiv:1708.07747*, 2017.
- [22] H. Sandhawalia and H. Jégou, "Searching with expectations," in *2010 IEEE International Conference on Acoustics, Speech and Signal Processing*. IEEE, 2010, pp. 1242–1245.
- [23] R. Balasubramonian, A. B. Kahng, N. Muralimanohar, A. Shafiee, and V. Srinivas, "Cacti 7: New tools for interconnect exploration in innovative off-chip memories," *ACM Transactions on Architecture and Code Optimization (TACO)*, vol. 14, no. 2, pp. 1–25, 2017.
- [24] S. Sarangi and B. Baas, "Deepscaletool: A tool for the accurate estimation of technology scaling in the deep-submicron era," in *2021 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, 2021, pp. 1–5.
- [25] M. Douze, A. Guzhva, C. Deng, J. Johnson, G. Szilvasy, P.-E. Mazaré, M. Lomeli, L. Hosseini, and H. Jégou, "The faiss library," 2024.
- [26] "Cuda implementation of hierarchical navigable small world graph algorithm." [Online]. Available: <https://github.com/js1010/cuhns>
- [27] S. Beamer, K. Asanović, and D. Patterson, "The gap benchmark suite," *arXiv preprint arXiv:1508.03619*, 2015.